

The main goal of this project was to **develop a simple chatbot system** capable of understanding user queries and responding appropriately using **rule-based or intent-based logic**.

This chatbot focuses on identifying user *intents* through predefined keywords and generating suitable responses.

## Methodology & Steps Performed

### Step 1: Define Intents and Example Queries

I created a set of **intents** such as:

- greeting
- goodbye
- thanks
- time
- weather

Each intent contains:

- **Keywords** (e.g., “hello”, “bye”, “thanks”)
- **Responses** (e.g., “Hi there!”, “Goodbye!”, “You’re welcome!”)

This forms the *knowledge base* of the chatbot.

### Step 2: Preprocessing User Input

I applied text preprocessing to make matching consistent:

- Converted text to **lowercase**
- Removed **punctuation** using regex (`re.sub(r'^\w\s]', '', text)`)
- **Tokenized** the text into words

This ensures that “Hello!”, “HELLO”, or “hello” are all treated the same way.

### Step 3: Intent Detection

Implemented a `get_intent()` function that:

- Compares the user’s processed words with keywords in each intent.
- Returns the intent name if a match is found.
- Returns “unknown” if no match is found.

This allows the chatbot to map user messages like “Hi” or “Good morning” to the *greeting* intent.

#### Step 4: Response Generation

Built a `generate_response()` function to:

- Choose a random appropriate response for matched intents.
- Handle special intents like “time” dynamically (e.g., showing current time).
- Use fallback responses for unrecognized inputs.

#### Step 5: Fallback Handling

Added a set of **fallback responses** such as:

“Hmm, I’m not sure about that.”

“Could you say that another way?”

“I didn’t quite catch that. Try rephrasing!”

This makes the chatbot sound more natural and prevents repetitive error messages.

#### Step 6: Testing the Chatbot

Tested the chatbot with varied queries:

| User Input           | Detected Intent | Chatbot Response                                                           |
|----------------------|-----------------|----------------------------------------------------------------------------|
| Hello                | greeting        | “Hi there! How can I help you?”                                            |
| What’s the time?     | time            | “The current time is 14:33:12.”                                            |
| Is it going to rain? | weather         | “I can’t fetch live weather data right now, but it looks like a good day!” |
| Thanks               | thanks          | “You’re welcome!”                                                          |

|     |         |                              |
|-----|---------|------------------------------|
| Bye | goodbye | "Goodbye! Have a great day!" |
|-----|---------|------------------------------|

### Step 7: Saving the Script

Saved the final chatbot as:

rule\_based\_chatbot.py

This makes it reusable and executable outside the Colab notebook.

### 3. Why I Chose the Rule-Based / Intent-Based Model

I chose this model because:

- It is **simple and interpretable** — each rule and response is clearly defined.
- It does **not require large datasets or ML training**.
- It is **lightweight**, making it ideal for beginner-level chatbot projects.
- It provides a **strong foundation** to later extend into an ML-based or NLP-based chatbot.

This model is particularly suited for structured or FAQ-style conversations where responses are predictable.

### 4. Results and Observations

- The chatbot correctly recognized and responded to most basic user inputs.
- It handled **unknown or unexpected queries** smoothly using fallback responses.
- Preprocessing significantly improved text matching accuracy.
- The chatbot's conversational tone became more natural through random response selection.

### 5. Conclusion

This project successfully implemented a **Rule-Based / Intent-Based Chatbot** in Python. It demonstrates how basic **Natural Language Processing (NLP)** and **pattern matching** can be used to simulate human-like conversation.

### Key Takeaways:

- Keyword-based intent detection works effectively for simple domains.
- Preprocessing ensures better input handling.
- Fallbacks make the conversation flow naturally.
- The approach provides a good base for upgrading to ML or transformer-based chatbots later.

### 6. Future Enhancements

- Integrate **machine learning** for automatic intent classification.
- Add **context tracking** to maintain conversation flow.
- Connect to APIs for real-time data (e.g., weather, news).
- Build a **web or voice-based interface** for user interaction.